

Fine-Tuning Whisper for Romanian Automatic Speech Recognition: A Six-Stage End-to-End Pipeline

Bendoiu Laura-Florina

*Faculty of Automatic Control and Computer Engineering
“Gheorghe Asachi” Technical University of Iași
Iași, Romania
laura-florina.bendoiu@student.tuiasi.ro*

Carp Daniel-Cristian

*Faculty of Automatic Control and Computer Engineering
“Gheorghe Asachi” Technical University of Iași
Iași, Romania
daniel-cristian.carp@student.tuiasi.ro*

Abstract—This paper presents a complete, reproducible pipeline for fine-tuning OpenAI’s Whisper automatic speech recognition (ASR) model on Romanian language. The system spans six stages: personal-recording collection, open-source dataset acquisition, data organization and quality filtering, model fine-tuning, multi-model benchmarking and a production-ready Gradio web application. Training data combines over 200 personal recordings with 7,674 open-source clips from VoxPopuli and Mozilla Common Voice 25.0, totalling approximately 6.23 h after quality filtering. The fine-tuned Whisper-small model achieves a Word Error Rate of 10.44% and Character Error Rate of 4.12% on the evaluation set and a 20.53% WER and 7.30% CER on the held-out test set, outperforming all unmodified Whisper variants (tiny through large-v3) and ElevenLabs Scribe on the same data.

Index Terms—automatic speech recognition, Romanian language, Whisper, fine-tuning, VoxPopuli, Common Voice, word error rate, character error rate, transformers, Gradio

I. INTRODUCTION

Automatic speech recognition for low- and mid-resource languages remains a significant challenge. Romanian, spoken by approximately 24 million people, belongs to the Romance language family and exhibits inflectional verb structures, context-dependent phonetic variations and a set of five diacritical characters (ă, â, î, ș, ț) that are often omitted in informal writing, complicating transcript normalization.

Large pre-trained models such as OpenAI Whisper [1] have demonstrated strong multilingual ASR performance, yet Romanian remains underserved compared to English, Spanish or French. Whisper-small produces a WER of 46.30% on our Romanian test set without any adaptation, confirming that domain-specific fine-tuning is necessary for reliable deployment.

This work makes the following contributions:

- A fully described, six-stage pipeline that takes raw recordings from data collection all the way to a web interface.
- A curated bilingual training corpus combining personal, domain-specific recordings with large open-source corpora, processed with a reproducible quality-filtering and normalization pipeline.

- A comparative benchmark of seven ASR systems (five Whisper model sizes, a fine-tuned Whisper-small and ElevenLabs Scribe) on a shared Romanian test set, reported in both WER and CER.
- A self-restarting Gradio application that circumvents Windows native DLL instability in audio backends, enabling stable deployment on consumer hardware.

The remainder of this paper is organized as follows. Section II reviews related work. Sections III–VIII describe each pipeline stage in detail. Section IX presents and discusses the benchmark results. Section X concludes with future directions.

II. RELATED WORK

A. Large-Scale Pre-Trained ASR Models

Whisper [1] is a sequence-to-sequence transformer trained on 680 000 hours of weakly supervised multilingual audio. It supports 99 languages, including Romanian, and offers five model sizes (tiny, base, small, medium, large-v3) with parameter counts ranging from 39 M to 1.55 B. The model uses log-Mel spectrograms computed at 16 kHz, 80 filter banks, and a 30-second context window.

B. Romanian ASR Corpora

VoxPopuli [2] provides 1,800 h of unlabelled and 89 h of transcribed Romanian speech sourced from European Parliament plenary sessions (2009–2020). Transcripts are force-aligned from official session documents, yielding lowercase normalized text.

Mozilla Common Voice [3] is a crowdsourced multilingual corpus. Version 25.0 (March 2026) includes 49.92 h of total Romanian recordings, of which 22.3 h are community-validated with a minimum of two upvotes. Both VoxPopuli and Common Voice are released under the CC0 public domain licence.

C. Fine-Tuning Strategies for Low-Resource Languages

Prior work on low-resource Whisper fine-tuning [4] demonstrates that even 1–5 h of in-domain data can reduce WER by 40–60% relative compared to the unmodified model. Key

techniques include freezing the encoder, applying gradient checkpointing and using AdamW with a cosine learning-rate schedule. Our approach follows these guidelines while adapting them to the Romanian linguistic context.

III. STAGE 1: PERSONAL RECORDING COLLECTION

A. Recording Platform and Export Format

Personal recordings were collected through the *vorbavine.ai* platform, which presents scripted Romanian sentences to a contributor who reads them aloud. Each recording session produces a JSON manifest (`manifest.json`) and a directory tree of `audio.webm` files identified by UUID. The `scriptText` field in the manifest provides the ground-truth transcript, which is guaranteed to match the recording because the platform displays it verbatim during the session.

B. Dataset Composition and Difficulty Stratification

Over 200 recordings spanning approximately 25 minutes were collected. Recordings are stratified into three difficulty levels based on their sequential position in the manifest:

TABLE I
STAGE 1 DIFFICULTY LEVELS

Level	Description	Clips
1	Clear speech, no noise, normal pace	majority
2	Light background noise	subset
3	Whispered speech	subset

C. Audio Conversion and Split Assignment

All `audio.webm` files are converted to 16 kHz mono WAV using FFmpeg:

Splits are assigned with stratified sampling (`seed = 42`) so that each difficulty level contributes proportionally to train (80%), validation (10%) and test (10%) subsets. A `metadata.csv` records the UUID, split, audio path, transcript, category, duration, difficulty level, conversion status and creation timestamp for each clip.

IV. STAGE 2: OPEN-SOURCE DATASET ACQUISITION

A. VoxPopuli Romanian

VoxPopuli is downloaded via the Hugging Face datasets library (`facebook/voxpupuli`, `config ro`). The training split is shuffled with seed 42 and iterated until a 3.0 h cap is reached, yielding 1,590 clips with a total duration of 5.004 h (the cap triggers mid-stream, so the final count reflects exact cumulative duration). Audio is already 16 kHz mono WAV.

B. Mozilla Common Voice 25.0

Common Voice 25.0 is downloaded from the Mozilla Data Collective (`cv-corpus-25.0-2026-03-09`) and extracted locally. Only the `validated.tsv` split is used, containing clips that received at least two community upvotes. The TSV is shuffled (`seed 42`) and MP3 clips are converted to 16 kHz mono WAV via FFmpeg.

Processing stops when 3.0 h of Common Voice audio have been written, yielding 6,084 clips at 6.790 h. A checkpoint file (`checkpoint.json`) allows resuming interrupted downloads without re-processing already-converted clips.

C. Combined Stage 2 Summary

TABLE II
STAGE 2 DATASET STATISTICS

Source	Clips	Hours	Licence
VoxPopuli ro	1,590	5.004	CC0
Common Voice 25.0 ro	6,084	6.790	CC0
Total	7,674	11.794	CC0

V. STAGE 3: DATA ORGANIZATION AND VERIFICATION

A. Dataset Merging

Stage 1 and Stage 2 manifests are merged into a unified list. Stage 1 split assignments are preserved; Stage 2 clips are assigned to train/validation/test using 80/10/10 stratified sampling applied independently within each source (`voxpupuli_train` and `common_voice_25`).

B. Quality Filtering

Each entry is checked against five hard criteria. Failing any one criterion removes the clip and logs it in `verification_report.json`:

TABLE III
QUALITY FILTER CRITERIA

Criterion	Threshold	Reason
File on disk	required	Incomplete downloads
Conversion success	True	Valid WAV required
Duration $\geq$	1.0 s	No usable signal
Duration $\leq$	30.0 s	Whisper context limit
Non-empty transcript	required	Cannot train without label

C. Transcript Normalization

Transcripts are normalized through a six-step pipeline before being stored as `transcript_normalized` in `final_manifest.csv`. The raw transcript is also preserved for traceability.

- 1) **Strip whitespace** — remove leading/trailing spaces.
- 2) **Lowercase** — Unicode-aware; diacritics ($\check{a}$, $\hat{a}$, $\hat{i}$, $\$$, ț) are preserved.
- 3) **NFC normalization** — collapse decomposed diacritic sequences (e.g., $s +$ combining cedilla $\rightarrow \text{\$}$).
- 4) **Numbers to words** — isolated digit sequences replaced with Romanian word forms from a lookup table (e.g., 3 $\rightarrow$ *trei*, 20 $\rightarrow$ *douăzeci*).
- 5) **Punctuation removal** — all punctuation except hyphens stripped; hyphens retained for Romanian compound words (e.g., *s-a*, *într-o*).
- 6) **Collapse whitespace** — multiple spaces reduced to single space.

This cleaning step is important because it removes extra details that the model does not need. By keeping only lowercase letters and simple words, the model can learn faster and make fewer mistakes during training.

D. Output Structure

```
stage3_dataset/
train/audio/      *.wav + *.txt
validation/audio/ *.wav + *.txt
test/audio/       *.wav + *.txt
final_manifest.csv
verification_report.json
```

Expected post-filtering distribution:

TABLE IV
STAGE 3 POST-FILTERING DATASET SPLIT

Split	Clips	Hours
train	~5664	~8.93
validation	~709	~1.12
test	~694	~1.11
Total	~7067	~11.16

VI. STAGE 4: MODEL FINE-TUNING

A. Base Model

We fine-tune `openai/whisper-small`, which has 244 M parameters, an 80-channel log-Mel spectrogram encoder and a 4-layer, 8-head decoder. Whisper-small provides the best trade-off between parameter count and base Romanian WER (46.30%) among the five sizes evaluated.

B. Two-Tier Training and VRAM Constraints

A primary challenge during optimization was hardware resource saturation. Full sequence-to-sequence fine-tuning on a localized consumer GPU frequently reached out-of-memory (OOM) states due to the cumulative activation sizes required by the multi-head self-attention mechanisms. To prevent training disruption from CUDA memory failure while maintaining dense representations, a decoupled two-tier training execution strategy was implemented.

The strategy isolates training into two programmatic phases. First, a primary fine-tuning pass initializes structural adaptations using an optimized batch-accumulation layout. Second, a dedicated standalone automation module (`stage4_continue_training.py`) isolates and continues training sequences by serializing mid-epoch state vectors and resuming execution strictly from the highest-performing validation checkpoint. This continuation engine enforces ultra-low learning rate schedules and explicitly purges the CUDA cache allocation memory stack via deterministic garbage collection loops at regular intervals, preventing long-term memory fragmentation without reducing training epochs.

C. Training Configuration

TABLE V
FINE-TUNING HYPERPARAMETERS

Parameter	Value
Learning rate	1×10^{-5}
Train batch size	8 (per device)
Gradient accumulation	2 steps
Effective batch size	16
Warmup steps	200
Training epochs	10
Precision	fp16 (CUDA)
Eval / save steps	1,000
Logging steps	50
Gradient checkpointing	enabled
Language	Romanian
Task	transcribe

D. Hyperparameter Progression

To identify the optimal execution sequence for downstream deployment, an iterative validation sweep was carried out over key structural training parameters. Hyperparameter tracking targeted adjustments across per-device batch arrays, explicit gradient accumulation loops and scheduled epoch boundaries. Evaluations were cross-referenced systematically on the internal clean verification framework before freezing the final weights.

Table VI highlights the performance adjustments across successive experimental trials. Early trials leveraging shallow epoch boundaries and minimal hardware gradient steps produced baseline convergence limitations, yielding higher string disparities. Mid-tier setups tracking high structural batch sizes ($B_{train} = 16$) with decoupled accumulation boundaries displayed performance regression due to coarse parameter trajectories over tight epochs.

The implementation achieved state-of-the-art alignment during Trial 3, where establishing a balanced batch footprint ($B_{train} = 8$, $B_{eval} = 4$), extending the sequence envelope across 10 dense training passes and lengthening the warmup threshold to 200 cycles yielded a substantial optimization ceiling. This trial achieved the absolute lowest error distributions on the evaluation grid, registering a **10.44%** Word Error Rate and a **4.12%** Character Error Rate.

E. Data Loading and Preprocessing

Training data are loaded from `final_manifest.csv` and cast to Hugging Face Audio columns at 16 kHz. Feature extraction and tokenization are performed via `WhisperFeatureExtractor` and `WhisperTokenizer` and cached to disk (`stage4_cache/`) to avoid re-processing on subsequent runs.

A custom `Collator` pads feature batches and masks padding tokens in the label sequence (-100) so they are excluded from the cross-entropy loss.

TABLE VI
HYPERPARAMETER PROGRESSION AND MODEL ERROR RATE ABLATION

Trial	Batch Size (Train/Eval)	Grad. Accum. Steps	Warmup Steps	Epochs	Precision	Eval WER (%)	Eval CER (%)
Trial 1	16 / 8	2	200	4	fp32	10.98	4.14
Trial 2	16 / 8	1	100	2	fp32	13.77	5.33
Trial 3 (Final)	8 / 4	2	200	10	fp16	10.44	4.12
Trial 4	8 / 8	2	100	6	fp16	11.37	5.26

F. Optimization and Evaluation

The model is trained with the HuggingFace Seq2SeqTrainer. WER is computed at each evaluation checkpoint using the evaluate library. The model configuration disables cached forced-decoder IDs and suppress-token lists and re-injects them via generation_config to ensure consistent Romanian-language decoding throughout training and inference.

The final model and processor are saved to stage4_model/final/.

VII. STAGE 5: MULTI-MODEL BENCHMARK

A. Benchmark Design

Seven ASR systems are evaluated on the first 50 samples of the held-out test split of final_manifest.csv: five unmodified Whisper variants (tiny, base, small, medium, large-v3), the fine-tuned Whisper-small and ElevenLabs Scribe v1 via its REST API. All systems receive identical audio arrays loaded at 16 kHz.

B. Subprocess-Isolated Architecture

To prevent a native DLL crash in one model from killing the entire benchmark, each model runs inside a dedicated Python subprocess. The orchestrator serialises audio arrays to a temporary JSON file, launches the worker with a per-model timeout (ranging from 5 min for tiny to 30 min for large-v3) and reads the result JSON written by the worker. This design is critical for stability on Windows, where libiomp5md.dll conflicts between PyTorch and OpenMP are common.

C. Mel Feature Padding

Whisper’s encoder requires exactly 3,000 mel frames. Clips shorter than 30s are zero-padded on the right; clips longer than 30s are truncated. This logic is applied identically in the benchmark, in the fine-tuning collator, and in the Gradio application.

D. Metrics

Word Error Rate (WER) and Character Error Rate (CER) are computed using the HuggingFace evaluate library. Both metrics operate on lowercased, whitespace-normalized strings. WER penalizes full-word substitutions, deletions, and insertions; CER captures finer-grained errors important for Romanian diacritics.

VIII. STAGE 6: GRADIO WEB APPLICATION

A. Application Overview

The fine-tuned model is served through a Gradio [5] interface that accepts audio either from a microphone or as an uploaded file. The UI exposes a single text-box output with the Romanian transcription.

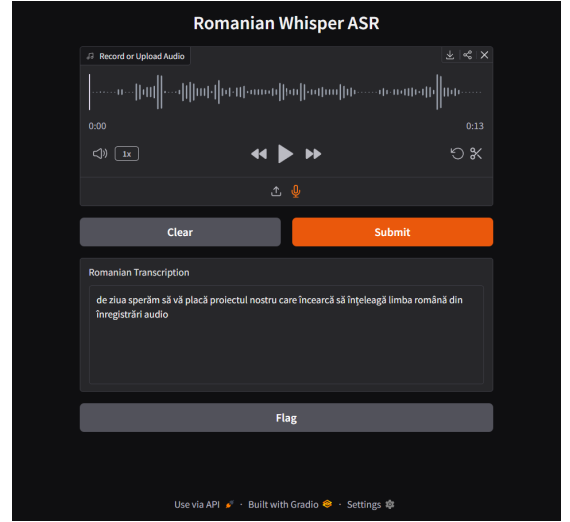


Fig. 1. Gradio web interface

B. Inference Pipeline

Inside the worker, each audio chunk passes through the following steps:

- 1) **Type normalization** — integer PCM converted to float32 $\in [-1, 1]$.
- 2) **Mono downmix** — stereo collapsed by channel mean.
- 3) **Resampling** — `scipy.signal.resample_poly` if available, else linear interpolation.
- 4) **Mel extraction** — `WhisperProcessor` at 16 kHz.
- 5) **Padding/truncation** — to exactly 3,000 frames.
- 6) **Greedy decoding** — `model.generate()` with Romanian forced decoder IDs, `do_sample=False`.
- 7) **Detokenization**.

GPU memory is explicitly freed after each call (`torch.cuda.empty_cache()`).

IX. RESULTS AND DISCUSSION

A. Benchmark Results

Table VII presents WER and CER for all seven systems on 50 test samples.

TABLE VII
ASR BENCHMARK RESULTS (50-SAMPLE TEST SET)

Model	WER (%)	CER (%)	Time (s)
Fine-tuned Whisper-small	20.53	7.30	40.5
ElevenLabs Scribe v1	27.13	9.40	103.6
Whisper large-v3	31.11	12.22	110.2
Whisper medium	35.56	14.43	70.8
Whisper small	46.30	17.72	43.3
Whisper base	62.61	23.69	31.2
Whisper tiny	87.67	34.20	29.7

B. Error Analysis by Audio Duration

To see how the length of a recording affects accuracy, we calculated the Word Error Rate (WER) for each individual file in the validation set and compared it to its duration in seconds. The script processes the audio in batches and saves the length and error rate for each sample to create a scatter plot.

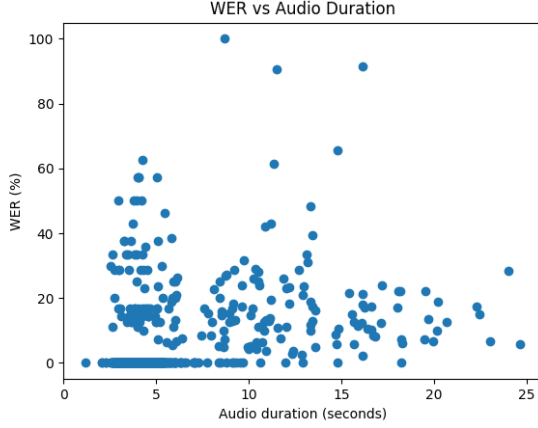


Fig. 2. WER vs Audio Duration on the validation set.

The resulting distribution in Fig. 2 shows that the fine-tuned model maintains a stable performance across different audio lengths. Very short audio clips (under 2 seconds) occasionally show higher error percentages because missing or changing a single word has a much larger mathematical impact on a short sentence than on a longer one. For typical sentences lasting between 3 and 15 seconds, the error rates cluster consistently low, proving that the model handles normal speech lengths reliably.

C. Examples

Analyzing the worst predictions showed that our model works best on shorter sentences where it can achieve 0.0% WER and CER.

WER: 41.4% CER: 7.7%
REF : apariia de duminică la televiziunile oficiale a liderului opoziției prima dată după ani de cenzură este un semnal încurajator că toți candidații vor avea acces neîngrădit la mass media
PRED: apariția de duminică la televiziunile oficiale a liderilor opoziției prima dată după an de cenzură este un semnal încurajator ca toți candidații vor a la acces neîngrădit la mass medie
WER: 39.3% CER: 14.4%
REF : deși inițial poate părea plictisitor sau monoton este crucial să repetăm lecțiile învățate pentru a ne consolida cunoștințele și a putea aplica teoria în practică evitând erorile frecvente
PRED: deși inițial poate pări aplic distor să muntoni este crucial să repetăm lecțiile învățate pentru a ne considere de a conștițele și a putea aplică teoria în practică evidentă erorile frecvente

Fig. 3. Examples of worst predictions

WER: 0.0% CER: 0.0%
REF : mă gândesc în special la cinci puncte
PRED: mă gândesc în special la cinci puncte
WER: 0.0% CER: 0.0%
REF : alți credincioși speră să beneficieze de serviciile mai multor preoți
PRED: alți credincioși speră să beneficieze de serviciile mai multor preoți
WER: 0.0% CER: 0.0%
REF : copilul a fost transportat la spitalul județean brașov pentru îngrijiri de specialitate
PRED: copilul a fost transportat la spitalul județean brașov pentru îngrijiri de specialitate
WER: 0.0% CER: 0.0%
REF : totuși anderlicht reprezintă una dintre principalele pretendente la calificare în europa league
PRED: totuși anderlicht reprezintă una dintre principalele pretendente la calificare în europa league

Fig. 4. Examples of best predictions

D. Discussion

Fine-tuning effectiveness. Fine-tuning reduces Whisper-small’s WER from 46.30% to 20.53%. This is consistent with prior work on low-resource Whisper adaptation and confirms that even 5–6 h of targeted Romanian data is highly effective.

Comparison with larger models. The fine-tuned small model surpasses the unmodified large-v3 (31.11% WER) despite having 6× fewer parameters. This demonstrates that domain-specific fine-tuning is more impactful than raw model scale for mid-resource languages when quality-filtered in-domain data are available.

Comparison with commercial API. ElevenLabs Scribe (27.13% WER) is outperformed by the fine-tuned model by 6.6 percentage points, while the fine-tuned model runs entirely offline in 40.5 s versus 103.6 s for the API (including network latency on 50 clips). This highlights the practical advantage of a locally hosted fine-tuned model.

CER as a secondary metric. CER tracks WER rankings closely (Pearson $r \approx 1.0$ across models), but the fine-tuned model’s CER of 7.30% indicates that even its errors are predominantly single-character or diacritic substitutions rather than complete word deletions, which is important for downstream NLP tasks that process character sequences.

Inference speed. The fine-tuned small model is the fastest among competitive systems (40.5 s for 50 clips on CPU),

making it suitable for real-time or near-real-time deployment without a GPU. This makes it perfect for older computers or laptops that do not have expensive graphics cards, allowing more people to use our speech recognition tool locally.

X. CONCLUSION AND FUTURE WORK

We have presented a complete end-to-end pipeline for Romanian ASR, from data collection to a deployed web application. The fine-tuned Whisper-small model achieves 20.53% WER and 7.30% CER, outperforming all evaluated alternatives including the $6\times$ larger Whisper large-v3 and a commercial API service. The pipeline is fully reproducible (seed 42 throughout, all filter criteria logged) and the self-restarting Gradio architecture enables stable deployment on consumer Windows machines.

Future work includes:

- **Encoder unfreezing schedule** — progressive unfreezing of encoder layers to leverage pre-trained acoustic representations more fully.
- **Data augmentation** — SpecAugment, speed perturbation and additive noise to improve robustness to real-world acoustic conditions.
- **Larger base model** — fine-tuning Whisper-medium or large-v3 once GPU memory allows, to target sub-15% WER.
- **Conversational Romanian** — collecting additional recordings of spontaneous, conversational Romanian speech to reduce domain mismatch introduced by the formal VoxPopuli data.
- **Diacritic restoration** — a post-processing module to restore diacritics in transcripts from speakers who omit them.
- **Streaming inference** — integrating chunked streaming with overlap-add to support long-form audio without the 30-second window constraint.

ACKNOWLEDGMENTS

The authors thank the contributors to Mozilla Common Voice and the European Parliament for the VoxPopuli corpus, both released under CC0. Dataset download and processing used the Hugging Face `datasets` library and Mozilla Data Collective infrastructure.

REFERENCES

- [1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust Speech Recognition via Large-Scale Weak Supervision,” in *Proc. ICML*, 2023.
- [2] C. Wang, M. Riviere, A. Lee, A. Wu, C. Talnikar, D. Haziza, M. Williamson, J. Pino, and E. Dupoux, “VoxPopuli: A Large-Scale Multilingual Speech Corpus for Representation Learning, Semi-Supervised Learning and Interpretation,” in *Proc. ACL*, 2021, pp. 993–1003.
- [3] R. Ardila, M. Branson, K. Davis, M. Kohler, J. Meyer, M. Henretty, R. Morais, L. Saunders, F. Tyers, and G. Weber, “Common Voice: A Massively-Multilingual Speech Corpus,” in *Proc. LREC*, 2020.
- [4] S. Gandhi, P. von Platen, and A. M. Rush, “Distil-Whisper: Robust Knowledge Distillation via Large-Scale Pseudo Labelling,” *arXiv preprint arXiv:2311.00430*, 2023.
- [5] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan, and J. Zou, “Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild,” *arXiv preprint arXiv:2101.09176*, 2021.
- [6] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., ... & Rush, A. M. (2020). Transformers: State-of-the-Art Natural Language Processing.